In-Depth Analysis of Iterative Sorting Algorithms

1. Bubble Sort

Definition & Approach

Bubble Sort is a simple comparison-based algorithm where each pair of adjacent elements is compared and swapped if they are in the wrong order. This cycle repeats until the entire list is sorted.

Steps:

1. Start at the beginning of the array.
2. Compare current element with the next element.
3. If current > next, swap them.
4. Move to the next pair and repeat until the end of the array.
5. After one full pass, the largest element is at the end.
6. Repeat the process for (n-1) elements.

Pseudocode:

```
procedure bubbleSort(A : list of sortable items)
    n = length(A)
    for i from 0 to n-1:
        for j from 0 to n-i-1:
            if A[j] > A[j+1]:
                swap(A[j], A[j+1])
```

Example Walkthrough

Array: [5, 1, 4, 2]. **Pass 1:** Compare (5,1) -> swap [1,5,4,2]. Compare (5,4) -> swap [1,4,5,2]. Compare (5,2) -> swap [1,4,2,5]. 5 is now in place.

Advantages: Extremely easy to implement, detects if array is already sorted, space efficient (O(1)).

Disadvantages: Very slow ($O(n^2)$), high number of swaps.

Use Cases: Useful when the input is nearly sorted or for teaching fundamental sorting logic.

2. Selection Sort

Definition & Approach

Selection Sort works by repeatedly finding the minimum element from the unsorted part and putting it at the beginning.

Steps:

1. Maintain two subarrays: sorted (left) and unsorted (right).
2. Find the minimum element in the unsorted subarray.
3. Swap this minimum with the first element of the unsorted subarray.
4. Move the boundary of the sorted subarray one step to the right.

Pseudocode:

```
for i from 0 to n-1:
    min_index = i
    for j from i+1 to n:
        if A[j] < A[min_index]:
            min_index = j
    swap(A[i], A[min_index])
```

Example Walkthrough

Array: [64, 25, 12, 22]. Find min in [64, 25, 12, 22] which is 12. Swap 12 with 64 -> [12, 25, 64, 22]. Repeat.

Advantages: Guaranteed $O(n^2)$ regardless of input order, minimum number of swaps (max n swaps).

Disadvantages: Not stable, inefficient for large lists.

Use Cases: Systems where write operations are costly (e.g., writing to Flash memory or EEPROM).

3. Insertion Sort

Definition & Approach

Insertion Sort builds the final sorted array one element at a time by taking elements from the input and inserting them into their correct position in the sorted sub-list.

Steps:

1. Start from the second element (index 1).
2. Compare the current element (key) with the element before it.
3. If key is smaller, shift the previous element to the right.
4. Continue shifting until a smaller element is found or the beginning is reached.
5. Insert the key into the correct spot.

Pseudocode:

```
for i from 1 to n:
    key = A[i]
    j = i - 1
    while j >= 0 and A[j] > key:
        A[j+1] = A[j]
        j = j - 1
    A[j+1] = key
```

Example Walkthrough

Array: [12, 11, 13, 5, 6]. Key=11. 11 < 12, shift 12 -> [11, 12, 13, 5, 6]. Next Key=13, no shift needed.

Advantages: Very fast for small data, adaptive (fast for nearly sorted), stable.

Disadvantages: Inefficient for very large, unsorted data.

Use Cases: Real-world scenarios like Timsort (used in Python and Java) use it for small slices of data.